



GitHub Copilot CLI: AI at the Point of Work

Close Every Gap Between You and the Work

Think anywhere. Steer anywhere. Learn always. Act autonomously.

"How do you bring AI to where the work actually is?"

Developers spend ~70% of their day outside VS Code — in terminals, on servers, and in CI pipelines.

AI that follows the work closes gaps the editor never could.

Software Developers

Debug and build where the code runs, not just where it lives



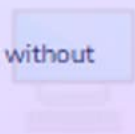
DevOps Engineers

Triage CI failures and audit containers from any device



CLI Power Users

Steer AI sessions across machines without leaving the terminal



~70% Outside the IDE

Deployments, debugging, log analysis — all outside VS Code (Atlassian DevEx 2025)

8 → 2 Attempts

Plan Mode cuts Docker debugging cycles from 8 trial-and-error attempts to 2

Any Device, Any Machine

A QR code puts AI on your production server — steer it from your phone in 60 seconds

PART 1

Think Before You Build

Plan Mode, Rubber Duck, and four operating modes

Understand intent before any code runs — stop guessing, start asking

PART 2

Work From Anywhere

Phone steering, --remote, and CI/CD automation

AI follows the problem wherever it lives — steer from any device

PART 3

Sessions That Learn

Auto-compaction, /rewind, /chronicle, live bridge

Sessions carry work forward, recover backward, and evolve your instructions

PART 4

Scale Beyond Yourself

/fleet, cloud delegation, scheduled execution

From one terminal to parallel agents acting on a schedule you defined

Click any section to jump directly there

Think Before You Build

Gap closed: Intent. Plan Mode clarifies before code runs — Rubber Duck reviews before it ships.



Plan Mode

Ask first — catch wrong assumptions before any code runs



Rubber Duck

Cross-model review catches what one reasoning path misses



Four Modes

Interactive, Plan, Programmatic, Remote — each closes a gap

Docker debug: 45 min, 8 attempts (manual)
↳ 8 min, 2 attempts — Plan Mode asked first

The IDE Only Sees Code

The terminal sees everything else

~70%

of developer time is spent outside the IDE

Atlassian State of Developer Experience 2025



Container logs & process state

Runtime crashes live in what's running — not in the source files



Network & environment variables

Connectivity, secrets validation, and env drift are diagnosed at the shell



CI/CD pipelines & build failures

Test results, deployment artifacts, and ephemeral runner state live here



Log analysis & incident response

Server logs stay on the server — the terminal is where forensics happen



An AI in the terminal has context the editor never sees — logs, environment, and running services.

Traditional Debugging vs Plan Mode

From trial-and-error to collaborative strategy

✗ Traditional Workflow

- 1 Make request
Describe what you need
- 2 AI generates solution
Assumes intent, one interpretation
- 3 You review and fix
Wrong assumptions surface late
- 4 Repeat until it works
Average: 8 attempts

🔄 8 attempts — average debug cycles before success

✓ With Plan Mode

- 1 Make request
Describe what you need
- 2 AI asks questions
Clarifies intent before acting
- 3 Collaborate on plan
Review the strategy before any code
- 4 Execute with confidence
Ambiguity resolved upfront

✓ 2 targeted attempts — Plan Mode closed the gap

8 debugging attempts → 2 with Plan Mode

Plan Mode + Rubber Duck in Action

Root cause in 8 minutes — then Rubber Duck reviews the fix

```
$ copilot
> "Debug why the backend container won't start"
🤖 Copilot (Plan Mode):
  Before I start, a few questions:
  1. Check docker-compose config, container logs, or both?
  2. Include environment variable analysis?
> "Start with logs, then check config if needed"

✅ Found: Port 5000 mapped to 5001 in docker-compose.yml
🦆 Rubber Duck (review pass):
  Fix is correct. One additional risk:
  Check BACKEND_PORT env var – if it overrides the binding,
  the port change alone won't hold after a restart.
✅ Root cause in 8 min – cross-model review caught the env var risk
```

8 attempts, 45 min → 2 attempts, 8 min

Four Operating Modes



Interactive

Collaborative sessions with persistent context — for debugging, exploration, and design



Plan Mode

Shift+Tab: AI asks clarifying questions before writing code — human-driven and deliberate



Programmatic

`copilot -p` for headless CI/CD — single commands, structured output, no conversation state



Remote

`copilot --remote`: AI lives on the server; steer from any device via URL or QR code

Work From Anywhere

Gap closed: Geography. A phone controls a live production server — steer any session from any device.



Phone Steering

QR code puts you in control from any device in 60 seconds



--remote Flag

AI session lives on the server; steer from wherever you are



CI/CD Mode

copilot -p runs headless in pipelines — no human required

Production server. SSH required. You're at a desk (before)
↳ QR scan → live steering from any device

Phone Steering: 60 Seconds on a Production Server

No context. No preamble. A phone controls an AI session running on a live server.

```
$ ssh ops@prod-server-3.us-east-1 && copilot --remote
```

🔗 Remote session started:

Monitor and steer from:

<https://github.com/copilot/sessions/abc123>

[QR CODE – scan with GitHub Mobile]

Session persists via tmux. Public preview: 2026-04-13

> "focus on auth log errors from the last 10 minutes"

📱 Steered from phone →

JWT validation failing on token expiry

Affected: 3 of 12 auth service replicas

Root cause: clock drift > 5s on nodes us-east-1b/1c

✅ Diagnosed: NTP desync – auth degradation, 10-second fix from a phone

AI on the server. You on the phone. No SSH needed.

How --remote Works

The Session

Durable entity — not tied to any terminal

Persists across disconnects and device switches

`/resume` from any machine, any device

Context travels

conventions, history, and reasoning stay alive

--remote Flag

Start any session with `copilot --remote`

Generates URL + QR code instantly

GitHub.com or GitHub Mobile: approve tools, steer, inject prompts

`/remote mid-session`

enable remote access without restarting

Programmatic Mode and Permission Safety

.github/workflows/build.yml

```
- name: Analyze failure
  if: failure()
  run: |
    copilot -p "Analyze the build failure"
    --allow-tool shell(gh)
    --allow-tool shell(git)
    > analysis.txt
# Enterprise enforcement:
# permissions.disableBypassPermissionsMode: true
```

⚙️ Programmatic mode

copilot -p: headless single-command execution, structured output for CI/CD pipelines

🔑 Per-tool approvals

--allow-tool grants specific permissions; never use --yolo on shared or production environments

📖 Enterprise enforcement

disableBypassPermissionsMode: true prevents --yolo across all org-managed environments

Sessions That Learn

Gap closed: Memory. Sessions carry work forward, recover backward, and learn how you work.



Learn & Recover

Auto-compaction forward, /rewind backward — work never lost



/chronicle

Session history → standup, tips, smarter instructions



Live Bridge

Scheduled execution fires while you present — the proof

```
Kick off before the talk: /every 1m tell me I'm awesome  
↳ fires live on the bridge slide — you taught it, now it executes
```


Learn and Recover: The Trust Foundation



Auto-compaction

At 95% token limit, history compresses — important context persists, noise pruned



/rewind

Restore conversation and Copilot-modified files without Git — later edits preserved



Sessions Sidebar

Press < to open. n creates a session, x closes — switch concurrent work in the CLI



Live Tool Durations

/usage and per-MCP token visibility; timeline shows elapsed time for active tool calls

/chronicle Standup: Yesterday's Work in 30 Seconds

Session history becomes a standup report — zero effort, always accurate

```
$ copilot
> "/chronicle standup"
📄 Chronicle reviewing session history:
Yesterday:
• Debugged auth service JWT expiry (prod-server-3)
• Refactored session store — 3 files changed, tests passing
• Opened issue #4821: clock drift in us-east-1b
Today:
• Continue: /fleet test coverage for auth service
• Follow up: NTP config for us-east-1b/1c
✅ 30-second standup, zero prep — drawn from what actually happened
```

Built from real session history, not memory

Session History Becomes Intelligence



/chronicle `standup`

Generate yesterday's standup from session history — everyday value in 30 seconds



/chronicle `tips`

AI surfaces your own usage patterns and suggests smarter workflows for how you work



/chronicle `instructions`

Propose improvements to `.github/copilot-instructions.md` — the session learned from you

You Taught It How You Work

Start `/every 1m tell me I'm awesome` before the talk — watch it arrive live on this slide

```
$ copilot --experimental  
> "/every 1m tell me I'm awesome"
```

🕒 Scheduled:

Every 1 minute, indefinitely.
First run: 60 seconds from now.
Schedule visible in footer hint bar.

🔔 [Live, ~30 minutes later, during this slide]:

- ✅ You're awesome! Keep up the great work.
- ✅ you taught it how you work; now it executes

Pattern learned in S3 → executes in S4 without asking

Scale Beyond Yourself

Gap closed: Reach. Everything learned in S3 becomes the context that shapes every task here.



/fleet Fan-Out

Parallel subtasks, isolated contexts,
merged results



Cloud Delegation

& prefix: delegate to the cloud — both
tools stay free



Scheduled Prompts

/every and /after — recurring agent
execution

Pattern learned in S3 – sessions, /chronicle, /every
↳ now executes autonomously in S4

The Scale Ladder

Each step expands reach without surrendering control

Sidebar

Sessions Sidebar

Concurrent conversations — switch with < sidebar; n and x to manage



/worktree

Isolated Worktrees

Experimental: separate conversation in a new worktree — exploratory changes stay isolated



/fleet

Parallel Fan-Out

Decompose a plan into parallel subtasks — each subagent in its own context window



TARGET

& delegate

Cloud Execution

Delegate to GitHub's coding agent — terminal and IDE stay completely free



/worktree ≠ /pr worktree — local isolation vs reviewing an existing pull request

/fleet: Parallel Task Execution

copilot session

```
# After approving a plan in Plan Mode:  
/fleet implement all phases of this auth refactor
```

```
# Or directly:  
/fleet create tests:  
  unit (auth service)  
  integration (API layer)  
  e2e (login flow)
```

⚡ Parallel subtasks

Orchestrator decomposes the plan — subagents run in separate context windows simultaneously

🔒 Context isolation

Each subagent has its own window — no context pollution between parallel workstreams

📄 Results merged

Orchestrator collects all subagent outputs and merges them back automatically when complete

Scale as an Ecosystem



Scheduled Prompts

`/every` and `/after` turn the CLI into a recurring agent runner — requires `/experimental` on



Custom Agents

Create specialized agents in `~/.copilot/agents/` (user), `.github/agents/` (repo), or org-level



Plugins

Install community and team plugins from the marketplace — specialized domain capabilities

From Guessing to Steering

Before

- 8 trial-and-error attempts to fix one container failure
- Re-explain codebase conventions at every session start
- Physically at the terminal to diagnose server issues
- Long-running tasks block IDE and terminal until done

After

- 2 targeted attempts — Plan Mode + Rubber Duck ask first
- Repository memory carries conventions across all sessions
- Steer any session from your phone via --remote
- Cloud delegation frees both tools — get a PR when done

8 → 2

debugging attempts with Plan Mode

37 min

saved per complex debug session

any device

steer sessions via --remote

✓ What You Can Do Today

Today

- Install Copilot CLI and try an interactive session
- Press Shift+Tab to enable Plan Mode before debugging
- Run `/chronicle` standup after your next work session

This Week

- Add `--remote` to your staging server and steer via mobile
- Run `/fleet` on a large multi-step refactor or test task
- Use `/rewind` to recover from a wrong turn without Git

This Month

- Enable `/every` for a scheduled PR summary or pod health check
- Run `/chronicle` instructions to improve your team's AGENTS.md
- Delegate a 30+ minute task with `&` and get a PR while you work

🔑 Key Takeaway

Think anywhere. Steer anywhere. Learn always. Act autonomously.

 Official Documentation[About GitHub Copilot CLI](#)

Core concepts, architecture, and session model

[Use Copilot CLI](#)

Command syntax, Plan Mode, modes, and slash commands

[Install Copilot CLI](#)

Setup instructions for all platforms

[Running tasks in parallel with /fleet](#)

Fan-out orchestration and subagent management

[Steering a session remotely](#)

--remote flag, QR code, and multi-device patterns

[Using /chronicle session data](#)

Standup reports, usage tips, and instruction improvements

 Related Content

[Custom instructions for Copilot CLI](#)

 Recent Updates[Remote control CLI sessions on web and mobile](#)

Public preview announcement — 2026-04-13

[GitHub Copilot weekly releases: August 3](#)

CLI updates including /rewind, Sessions sidebar, and live tool durations



AI at the Point of Work

GitHub Copilot CLI: Close Every Gap Between You and the Work

8 → 2

debugging attempts with Plan Mode

60 sec

phone steering a live production server

/chronicle

session history → standup, tips, smarter instructions

Where in your workflow would AI at the terminal free the most time?

Interactive Mode — Agent Environment

<code>/init</code>	Initialize Copilot instructions for this repo
<code>/agent</code>	Browse and select from available agents
<code>/skills</code>	Manage skills for enhanced capabilities
<code>/mcp</code>	Manage MCP server configuration
<code>/plugin</code>	Manage plugins and plugin marketplaces
<code>/instructions</code>	View & toggle custom instruction files
<code>/env</code>	Show loaded instructions, MCPs, skills, agents

What this gives you

Everything that shapes the agent's behavior in this session — instructions, custom agents, skills, MCP servers, and plugins.

Start here

`/init` once per repo, then `/env` any time you wonder "what does the agent actually know right now?"

Interactive Mode — Models & Subagents

<code>/model</code>	Select AI model to use
<code>/autopilot</code>	Toggle autopilot mode
<code>/fleet</code>	Parallel subagent execution
<code>/delegate</code>	Send session to GitHub → create a PR
<code>/tasks</code>	View & manage tasks (subagents, shell)
<code>/sidekicks</code>	View running sidekick agents

Three execution gears

- Interactive — you approve each step
- Autopilot — agent runs until done
- Fleet — multiple subagents in parallel

Hand-off vs parallelize

`/delegate` ships the work to GitHub coding agent;
`/fleet` stays local and fans out subagents.

Interactive Mode — Code

<code>/plan</code>	Implementation plan before coding
<code>/diff</code>	Review the changes made in current di
<code>/pr</code>	Operate on pull requests for this branch
<code>/review</code>	Run code review agent on changes
<code>/ide</code>	Connect to an IDE workspace
<code>/lsp</code>	Manage language server configuration
<code>/terminal-setup</code>	Configure shift+enter for multiline

Plan → Code → Review

`/plan` drafts the approach, the agent codes, then `/diff` and `/review` let you inspect & refine before opening a PR.

IDE bridge

`/ide` pairs the CLI with an open editor — file edits open as native diffs and selections flow back to Copilot.

Interactive Mode — Session

<code>/resume</code>	Switch sessions (id / task id / name)
<code>/session</code>	View & manage sessions
<code>/rename</code>	Rename or auto-name current session
<code>/context</code>	Show context window token usage
<code>/usage</code>	Session usage metrics & stats
<code>/compact</code>	Summarize history to free context
<code>/memory</code>	Toggle cross-session fact recall
<code>/rewind</code>	Undo last turn + revert file changes
<code>/share</code>	Export to markdown / HTML / gist
<code>/copy</code>	Copy last response to clipboard
<code>/remote</code>	Toggle control from GitHub web / mobile

Context hygiene

Watch `/context` . When it's > 70% full, run `/compact` with focus instructions instead of starting over.

Sessions are persistent

All sessions live in `~/.copilot/session-store.db` . `/resume` picks any one up — even days later, by name or ID prefix.

Interactive Mode — Permissions

<code>/allow-all</code>	Enable all permissions (tools, paths, URLs)
<code>/add-dir</code>	Whitelist a directory for file access
<code>/list-dirs</code>	Show all allowed directories
<code>/cwd</code>	Change or show working directory
<code>/reset-allowed-tools</code>	Clear the list of allowed tools

Three permission surfaces

- Tools — what the agent can invoke
- Paths — what files it can touch
- URLs — what domains it can reach

Denial always wins

Deny rules override `/allow-all`. Safe pattern: allow broadly, deny specifically (e.g. `shell(git push)`).

Interactive Mode — Help & More

<code>/research</code>	Deep research via GitHub + web
<code>/chronicle</code>	Session history insights
<code>/changelog</code>	CLI version history (+ AI summary)
<code>/new</code>	Start a new conversation
<code>/clear</code>	Abandon session and start fresh
<code>/keep-alive</code>	Prevent system sleep during work
<code>/voice</code>	Dictation via Foundry Local
<code>/theme</code>	View or set color mode
<code>/streamer-mode</code>	Hide model names / quota (screen share)
<code>/experimental</code>	Enable / disable experimental features
<code>/feedback</code>	Provide feedback about the CLI
<code>/help</code>	Show help for interactive commands

Long-running work

`/research` spins up a deep-dive task; pair with `/keep-alive` so your laptop doesn't sleep mid-run.

Demo & screen-share kit

`/streamer-mode` hides quota and preview model names; `/theme` swaps light/dark to match your slides.

Non-Interactive Mode — Scriptable AI

One-shot prompt, exits after response

```
copilot -p "Fix the failing test in auth.test.ts" --allow-all
```

Quiet output – only agent response, no stats (great for piping)

```
copilot -p "Summarize this PR" --silent | pbcopy
```

Autopilot – continues autonomously until done

```
copilot -p "Migrate all API routes to use the new auth middleware" --allow-all --autopilot
```

Share session output as a GitHub gist

```
copilot -p "Audit this repo for security issues" --allow-all --share-gist
```

Structured JSON output for CI pipelines

```
copilot -p "Run tests and report failures" --allow-all --output-format json
```


Permissions — Granular Control

```
# Allow specific tool categories
copilot --allow-tool=write --allow-tool=shell

# Allow all git except push
copilot --allow-tool='shell(git:*)' \
--deny-tool='shell(git push)'

# Restrict to specific directories
copilot --add-dir ~/workspace --add-dir /tmp

# Allow GitHub API access
copilot --allow-url=github.com

# Enable everything – tools, paths, URLs
copilot --yolo

# Same as:
copilot --allow-all-tools \
--allow-all-paths \
--allow-all-urls

--allow-all shorthand for all three
--yolo alias for --allow-all

# Required for non-interactive (-p) mode
copilot -p "... " --allow-all-tools
```


`copilot init` — Bootstrap Your Repo

Analyzing codebase structure ...

Detecting build system: npm

Detecting test framework: jest

Detecting language: TypeScript

✓ Created `.github/copilot-instructions.md`

What it generates

Analyzes your repo with read-only tools and writes `.github/copilot-instructions.md` with:

- Build & test commands
- Coding conventions
- Project structure overview
- Technology stack details

Why it matters

Every future session in this repo automatically loads these instructions — no need to re-explain your project to Copilot.

`copilot login` — Authentication

```
# Interactive browser flow (default)
copilot login

# GitHub Enterprise Cloud (data residency)
copilot login --host https://example.ghe.com

# Headless / CI - use env var instead of login
COPILOT_GITHUB_TOKEN=github_pat_... copilot

# Token precedence order:
COPILOT_GITHUB_TOKEN
GH_TOKEN
GITHUB_TOKEN
```

Supported token types

- Fine-grained PATs (v2) with *Copilot Requests* permission
- OAuth tokens from GitHub Copilot CLI app
- OAuth tokens from GitHub CLI (gh) app

✗ Classic PATs (ghp_...) are not supported

Token storage

Stored in system credential store. Falls back to plain text in `~/.copilot/` if no credential store is available.

`copilot plugin` — Extend with Plugins

```
# Browse the default marketplace
copilot plugin marketplace browse copilot-plugin

# Install from GitHub repository
copilot plugin install github/my-plugin

# Install from local directory
copilot plugin install ./my-local-plugin

# List installed plugins
copilot plugin list

# Update / uninstall
copilot plugin update my-plugin
copilot plugin uninstall my-plugin
```

What plugins can add

- Additional skills & agents
- Git hooks
- MCP servers
- LSP servers

Built-in marketplaces

```
copilot-plugins github/copilot-plugins
awesome-copilot github/awesome-copilot
```

copilot help config — Key Settings

Behavior

model – AI model (gpt-5.4, claude-sonnet-4.6, ...)
autoUpdate – auto-download CLI updates (default: true)
memory – agentic cross-session fact recall (default: true)
keepAlive – prevent system sleep (off / on / busy)
experimental – enable experimental features
renderMarkdown – render markdown in terminal
respectGitignore – hide gitignored files from @ picker
includeCoAuthoredBy – add Co-authored-by to commits
compactPaste – collapse large pastes (>10 lines)
streamerMode – hide model names & quota (for screen sharing)

Terminal

mouse – enable mouse support in alt screen
beep – beep when attention required

Security

allowedUrls – pre-approved URL/domain list (supports *.domain)
deniedUrls – blocked domains (takes precedence)
trustedFolders – folders with persisted read/exec approval

Available models

claude-sonnet-4.6 claude-sonnet-4.5 claude-haiku-4.5
claude-opus-4.8 claude-opus-4.7 claude-opus-4.6
claude-opus-4.6-fast claude-opus-4.5
gpt-5.5 gpt-5.4 gpt-5.2
gpt-5.3-codex gpt-5.2-codex
gpt-5.4-mini gpt-5-mini

Teams

companyAnnouncements – messages shown in banner on startup

copilot help environment — Key Env Vars

Authentication

`COPILOT_GITHUB_TOKEN` – PAT / OAuth token (highest precedence)

`GH_TOKEN` – GitHub CLI token (second)

`GITHUB_TOKEN` – CI token (third)

`GH_HOST` – GitHub Enterprise hostname

`COPILOT_GH_HOST` – Copilot-only host (overrides `GH_HOST`)

Behavior

`COPILOT_ALLOW_ALL=true` – skip all confirmations

`COPILOT_MODEL` – override AI model

`COPILOT_AUTO_UPDATE=false` – disable auto-update

`COPILOT_HOME` – override `~/.copilot` directory

`COPILOT_OFFLINE=true` – no network (needs BYOK)

`COPILOT_CUSTOM_INSTRUCTIONS_DIRS` – extra `AGENTS.md` dirs

BYOK – Bring Your Own Key

`COPILOT_PROVIDER_BASE_URL` – custom API endpoint (activates BYOK)

`COPILOT_PROVIDER_TYPE` – openai / azure / anthropic

`COPILOT_PROVIDER_API_KEY` – API key

`COPILOT_PROVIDER_BEARER_TOKEN` – bearer token (takes precedence over API key)

`COPILOT_PROVIDER_WIRE_API` – completions / responses (use responses for GPT-5)

`COPILOT_PROVIDER_AZURE_API_VERSION` – Azure API version

Proxy & Output

`HTTP_PROXY` / `HTTPS_PROXY` – outbound proxy URL

`NO_PROXY` – bypass list (comma-separated)

`NO_COLOR` – disable color output

`PLAIN_DIFF=true` – disable rich diff

BYOK — Bring Your Own Key

```
# Ollama (local, no auth required)
COPILOT_PROVIDER_BASE_URL=http://localhost:11434/v1 \
COPILOT_MODEL=deepseek-coder-v2:16b \
copilot

# Custom OpenAI-compatible endpoint
COPILOT_PROVIDER_BASE_URL=https://my-api.example.com/v1 \
COPILOT_PROVIDER_API_KEY=sk- ... \
COPILOT_MODEL=gpt-4 \
copilot

# Azure OpenAI with deployment name
COPILOT_PROVIDER_TYPE=azure \
COPILOT_PROVIDER_BASE_URL=https://my.openai.azure.com \
COPILOT_PROVIDER_MODEL_ID=gpt-4 \
COPILOT_PROVIDER_WIRE_MODEL=my-gpt4-deployment \
copilot
```

Provider types

openai – default; covers Ollama, vLLM, Foundry Local
azure – Azure OpenAI (host URL only)
anthropic – Anthropic API

When to use

- Air-gapped or offline environments
- Enterprise contracts with specific model providers
- Local development with Ollama / Foundry Local
- Cost control with self-hosted models

`/fleet` — Parallel Subagent Execution

*Refactor auth module, add unit tests,
update API docs, run the linter.*

`/fleet`

✓ Plan — 4 subtasks identified

[subagent-1] Refactor auth module

[subagent-2] Write unit tests

[subagent-3] Update API documentation

[subagent-4] Run linter across repo

Running in parallel...

✓ [subagent-4] Linter: 3 warnings fixed

✓ [subagent-1] Auth refactored

✓ [subagent-3] 12 endpoints updated

✓ [subagent-2] 24 tests added, passing

✓ All subtasks complete

How it works

- Main agent acts as orchestrator
- Decomposes prompt into independent subtasks
- Runs in parallel, each with its own context
- Manages dependencies; merges results

Best used for

- Multi-file refactors + test generation
- Audits + fixes across modules
- Large dependency updates in monorepos

Power combos

Shift+Tab → plan → Accept + autopilot + `/fleet`
"Use GPT-5.3-Codex to refactor... "
"Use @test-writer to create tests... "

Where the Magic Lives — `~/.copilot/`

How Copilot remembers everything — in-session and between sessions

<code>agents/</code>	custom agents
<code>skills/</code>	skill packs
<code>ide/</code>	IDE bridge
<code>logs/</code> <code>session-state/</code>	in-session state
<code>pkg/</code>	CLI & auto-update
<code>restart/</code>	restart staging
<code>config.json</code>	settings
<code>permissions-*.json</code>	allow/deny rules
<code>session-store.db</code>	past sessions (SQLite)
<code>command-history-*.json</code>	prompt history

In-session

- `session-state/` — context, tasks, subagent state
- `ide/` — live editor bridge
- `logs/` — execution logs

Between-session

- `session-store.db` — all conversations, resumable
- `command-history-*.json` — up-arrow recall
- `permissions-*.json` — persistent approvals

Portable capabilities

- `agents/` & `skills/` follow you everywhere
- Override with `$COPILOT_HOME`